
Phân công code vấn đáp đồ án

X509Certificate

Phần việc chi tiết của thành viên

Nguyễn Trọng Phúc

*Lỗi mật mã, Root CA
và Phát hành chứng chỉ*

Tài liệu này mô tả chi tiết phần code, lý thuyết X.509 cần nắm, các test case và cách tích hợp cho phần việc **lỗi mật mã + Root CA + phát hành cert**, gắn trực tiếp với mã nguồn thực tế của đồ án trong các thư mục `src/core/` (`ca.py`, `cert_builder.py`, `csr.py`), `src/services/` (`ca_admin.py`, `csr_admin.py`, `csr_workflow.py`, `customer_keys.py`) và `src/ui/`.

Môn: Mã hoá và Ứng dụng (MHUD)

Học phần đồ án X.509 Certificate

Ngày biên soạn: 14/06/2026

Contents

1	Mục tiêu và phạm vi phần việc	2
1.1	Ảnh xạ khái niệm → file code → vai trò	2
2	Kiến trúc tổng thể và vị trí của thành viên trong hệ thống	2
2.1	Luồng phát hành chứng chỉ (sơ đồ)	2
2.2	Phân lớp module và quan hệ	4
3	Phần code cần làm (chi tiết)	4
3.1	Tạo Root CA self-signed (<code>ca_admin.generate_root_ca</code>)	4
3.2	Lưu Root CA encrypt-at-rest (<code>ca_admin.create_root_ca</code>)	5
3.3	Nạp Root CA để ký (<code>load_active_root_ca_with_key</code>)	6
3.4	Bản legacy core/ca.py (file & trust store)	7
3.5	Sinh keypair khách hàng (<code>customer_keys.generate_keypair</code>)	7
3.6	Tạo CSR theo PKCS#10 (<code>core/csr.build_csr</code>)	8
3.7	Customer nộp CSR (<code>csr_workflow.submit_csr</code>)	9
3.8	Phát hành cert (<code>csr_admin.approve_csr</code>)	10
3.9	Builder cert v3 (<code>cert_builder._build_end_entity_cert</code>)	11
3.10	Tham số cấp phát (<code>system_config</code>)	12
3.11	Toàn cảnh nhà máy sinh cert — <code>cert_builder.py</code>	13
3.12	Lớp UI của cụm	14
4	Cần nắm để vấn đáp	14
5	Test case và demo	16
5.1	Bảng test case chính	16
5.2	Đoạn unit test gợi ý	16
5.3	Lệnh chạy test	17
5.4	Kịch bản demo end-to-end	17
6	Tích hợp vào chương trình chung	18
7	Bổ sung sau rà soát: các hàm/feature dễ bị hỏi	18
7.1	Nộp CSR qua LAN & khái niệm khóa “public-only”	18
7.2	Các hàm CSR/CA/cấu hình còn lại	19
8	Checklist chuẩn bị vấn đáp	19

1 Mục tiêu và phạm vi phân việc

Theo bảng phân công chung, thành viên **Nguyễn Trọng Phúc** phụ trách **lỗi mật mã, Root CA và phát hành chứng chỉ**. Đây là phần *đứng đầu* trong vòng đời chứng chỉ: nó tạo ra gốc tin cậy (Root CA self-signed), sinh cặp khóa cho khách hàng, đóng gói yêu cầu cấp chứng chỉ (CSR theo chuẩn PKCS#10), và cuối cùng **phát hành** chứng chỉ X.509 v3 hợp lệ bằng cách dùng private key của Root CA để ký. Nói cách khác, cụm này *sản xuất* ra mọi vật phẩm mật mã mà các cụm khác (kiểm tra chuỗi, thu hồi CRL & OCSP) sẽ tiêu thụ về sau.

Phạm vi gồm bốn trục chính:

1. **Sinh khóa và chữ ký số** — RSA 2048/3072/4096, ký SHA-256 với padding PKCS#1 v1.5.
2. **Root CA** — tạo cert self-signed, mã hóa private key *encrypt-at-rest* (AES-256-GCM) trước khi lưu DB, rotate, publish ra trust store.
3. **CSR (PKCS#10)** — build/parse/verify; chứng minh quyền sở hữu khóa (proof of possession).
4. **Phát hành cert** — duyệt CSR, build cert end-entity v3 với đầy đủ extension (KeyUsage, EKU, SAN, CRLDP, AIA, SKI/AKI), ký bằng Root CA.

1.1 Ánh xạ khái niệm → file code → vai trò

Điểm cần nhớ khi mở bài: cụm này là *bên phát hành* (issuer side). Mọi thứ nó tạo ra đều có một chữ ký số làm gốc tin cậy: CSR được ký bằng khóa khách hàng (chứng minh sở hữu), cert được ký bằng khóa Root CA (chứng minh do CA phát hành), Root CA tự ký chính mình (vì nó là gốc, không có ai trên nó).

2 Kiến trúc tổng thể và vị trí của thành viên trong hệ thống

Hệ thống dùng mô hình **Root CA + Trust Store** hai tầng: một Root CA self-signed ký trực tiếp các cert end-entity (không có intermediate CA), và client tin cậy cert nhờ có Root CA trong trust store. Cụm của thành viên nằm ở *nửa trên* của vòng đời chứng chỉ.

2.1 Luồng phát hành chứng chỉ (sơ đồ)

(1) Admin: tạo Root CA self-signed `ca_admin.create_root_ca`

↓

Khái niệm	File / hàm code	Vai trò
Sinh keypair RSA + tạo cert	core/cert_builder.py	Hàm dựng cert X.509 v3, builder pattern của thư viện <code>cryptography</code> .
Root CA self-signed (production)	services/ca_admin.py _generate_root_ca()	Sinh Root CA, mã hóa key AES-GCM, lưu DB, rotate.
Root CA self-signed (legacy/lab)	core/ca.py load_or_create_issuer()	Bản cũ lưu cert/key ra file + trust store.
Sinh keypair khách hàng	services/customer_keys.py generate_keypair()	RSA + encrypt-at-rest, AAD bind theo id, BOLA guard.
CSR (PKCS#10)	core/csr.py	build_csr / parse_csr / verify_csr_signature.
Customer nộp CSR	services/csr_workflow.py submit_csr()	Validate domain/wildcard, ký CSR, lưu pending.
Duyệt và phát hành cert	services/csr_admin.py approve_csr()	Verify CSR → load Root CA → build cert → ký.
Tham số cấp phát	services/system_config.py	key_size / validity / algo / URL nhúng cert.
UI sinh/rotate Root CA	ui/admin/root_ca_view.py	Màn hình quản lý Root CA.
UI duyệt CSR → cấp cert	ui/admin/csr_queue_view.py	Queue duyệt, approve/reject.
UI khách hàng	ui/customer/my_keys_view.py ui/customer/csr_submit_view.py	Sinh keypair, gửi CSR.

(2) Customer: sinh keypair RSA `customer_keys.generate_keypair`



(3) Customer: tạo + ký CSR (PKCS#10) `csr.build_csr / csr_workflow.submit_csr`



(4) Admin: verify CSR → load Root CA → ký cert `csr_admin.approve_csr`



(5) Cert X.509 v3 phát hành `cert_builder.issue_cert_from_csr`

Tầng	Module	Trách nhiệm
UI	root_ca_view, csr_queue_view, my_keys_view, csr_submit_view	Thu thập input, gọi service, hiển thị kết quả, ghi audit.
Service	ca_admin, csr_admin, csr_workflow, customer_keys, system_config	Nghệp vụ + giao dịch DB (atomic, BOLA guard, race-check).
Core (mật mã)	cert_builder, csr, ca, encryption	Thao tác mật mã thuần (sinh khóa, ký, build cert, AES-GCM).
DB	root_ca, customer_keys, csr_requests, issued_certs, system_config	Lưu trữ bền vững.

2.2 Phân lớp module và quan hệ

Quy tắc thiết kế: tầng core/ chỉ làm mật mã *thuần*, không chạm DB; tầng services/ ghép mật mã với DB trong transaction; tầng UI không bao giờ tự ký/giải mã mà luôn gọi xuống service. Nhờ vậy mỗi tầng test được độc lập.

3 Phần code cần làm (chi tiết)

3.1 Tạo Root CA self-signed (`ca_admin._generate_root_ca`)

Root CA là điểm neo tin cậy. Vì nó là gốc nên *tự ký* chính mình: `subject == issuer`, và chữ ký được tạo bằng *chính private key của nó*. Đây là điểm khác biệt cốt lõi so với cert end-entity (do Root CA ký).

```

1 def _generate_root_ca(common_name, key_size, validity_days):
2     key = rsa.generate_private_key(public_exponent=65537, key_size=key_size)
3     # subject == issuer → đặc trưng của cert self-signed
4     subject = issuer = x509.Name([
5         x509.NameAttribute(NameOID.COUNTRY_NAME, "VN"),
6         x509.NameAttribute(NameOID.ORGANIZATION_NAME, "X509 Demo"),
7         x509.NameAttribute(NameOID.COMMON_NAME, common_name),
8     ])
9     now = datetime.now(timezone.utc)
10    cert = (
11        x509.CertificateBuilder()
12        .subject_name(subject).issuer_name(issuer)
13        .public_key(key.public_key())
14        .serial_number(x509.random_serial_number())
15        .not_valid_before(now - timedelta(minutes=1))

```

```

16         .not_valid_after(now + timedelta(days=validity_days))
17         # ... các extension bên dưới ...
18         .sign(key, hashes.SHA256())    # ← TỰ KÝ bằng chính khóa của nó
19     )
20     return cert, key

```

Hai extension định danh “đây là CA”:

```

1  .add_extension(
2      x509.BasicConstraints(ca=True, path_length=0), critical=True,
3  )
4  .add_extension(
5      x509.KeyUsage(
6          digital_signature=True, key_cert_sign=True, crl_sign=True,
7          key_encipherment=False, content_commitment=False,
8          data_encipherment=False, key_agreement=False,
9          encipher_only=False, decipher_only=False,
10     ),
11     critical=True,
12 )
13 .add_extension(
14     x509.SubjectKeyIdentifier.from_public_key(key.public_key()),
15     critical=False,
16 )

```

Giải thích từng ý:

- `BasicConstraints(ca=True, path_length=0)`: xác nhận đây là CA và `path_length=0` giới hạn chuỗi — Root CA chỉ được ký trực tiếp end-entity cert, *không* được ký thêm CA con (intermediate). Đặt `critical=True` để client buộc phải hiểu và tuân thủ.
- `KeyUsage`: bật `key_cert_sign` (được ký cert) và `crl_sign` (được ký CRL) — đúng hai vai trò của một CA. Tắt `key_encipherment` vì khóa CA không dùng để mã hóa dữ liệu.
- `SubjectKeyIdentifier (SKI)`: vân tay của public key Root CA; cert con sẽ trở về SKI này qua `AuthorityKeyIdentifier (AKI)`, giúp client nối chuỗi nhanh và chính xác.

3.2 Lưu Root CA encrypt-at-rest (`ca_admin.create_root_ca`)

Private key của Root CA là tài sản nhạy cảm nhất hệ thống — lộ nó là lộ toàn bộ tin cậy. Vì vậy nó được **mã hóa AES-256-GCM** trước khi ghi DB, và việc ghi diễn ra atomic: deactivate Root CA cũ rồi insert cái mới trong cùng một transaction.

```

1  cert, key = _generate_root_ca(common_name, key_size, validity_days)
2  cert_pem = cert.public_bytes(serialization.Encoding.PEM)
3  key_pem = _serialize_private_key_pem(key)    # PKCS8 PEM, không password
4
5  # Encrypt-at-rest. AAD = b"root_ca" để bind ciphertext với loại record.

```

```

6  nonce, ct = encrypt_blob(key_pem, aad=ROOT_CA_AAD)
7  # ...
8  with transaction(db_path) as conn:
9      conn.execute("UPDATE root_ca SET is_active = 0 WHERE is_active = 1")
10     cur = conn.execute(
11         "INSERT INTO root_ca (common_name, serial_hex, cert_pem, "
12         " encrypted_private_key, gcm_nonce, ..., is_active) "
13         "VALUES (?, ?, ?, ?, ?, ..., 1)",
14         (common_name, serial_hex, cert_pem, ct, nonce, ...),
15     )
16     new_id = cur.lastrowid

```

- **Tại sao mã hóa:** kể cả khi file DB bị copy ra ngoài, kẻ tấn công cũng chỉ thấy ciphertext; không có master key thì không giải mã được private key.
- **AAD (Additional Authenticated Data)** = b"root_ca": dữ liệu bổ trợ được *xác thực* (không mã hóa) bởi GCM. Nếu ai đó lấy ciphertext của Root CA rồi cố giải mã trong ngữ cảnh khác (AAD khác), tag check sẽ thất bại. Nó “cột” ciphertext vào đúng loại bản ghi.
- **Atomic:** UPDATE ... is_active=0 và INSERT ... is_active=1 cùng một transaction → không bao giờ tồn tại trạng thái không có hoặc có hai Root CA active.

Khi **rotate** (sinh Root CA mới đè cái cũ), các cert đã phát hành vẫn mang chữ ký của Root CA *cũ*. Client sẽ FAIL verify cho đến khi admin re-issue cert mới. UI có cảnh báo đỏ về điều này (xem `root_ca_view`).

3.3 Nạp Root CA để ký (`load_active_root_ca_with_key`)

Đây là hàm *duy nhất* giải mã private key Root CA, chỉ gọi khi cần ký cert mới hoặc ký CRL.

```

1  def load_active_root_ca_with_key(db_path):
2      with conn_scope(db_path) as conn:
3          row = conn.execute(
4              "SELECT cert_pem, encrypted_private_key, gcm_nonce "
5              "FROM root_ca WHERE is_active = 1 LIMIT 1"
6          ).fetchone()
7      if row is None:
8          raise CAError("Chưa có Root CA active. Admin cần tạo Root CA trước.")
9      cert = x509.load_pem_x509_certificate(row["cert_pem"])
10     key_pem = decrypt_blob(
11         row["gcm_nonce"], row["encrypted_private_key"], aad=ROOT_CA_AAD,
12     )
13     key = serialization.load_pem_private_key(key_pem, password=None)
14     return cert, key

```

Lưu ý AAD truyền vào `decrypt_blob` phải đúng bằng AAD lúc encrypt (`ROOT_CA_AAD`), nếu không GCM tag check fail và raise lỗi — cơ chế chống nhầm/giả mạo ngữ cảnh.

3.4 Bản legacy core/ca.py (file & trust store)

`core/ca.py` giữ đường legacy/lab: lưu cert/key Root CA ra *file* thay vì DB, và publish ra trust store cho client. Hàm `load_or_create_issuer` “load nếu có, sinh nếu chưa”. Vì là đường lab nên private key *không* encrypt-at-rest; bù lại nó siết quyền file ở mức OS.

```

1  def load_or_create_issuer(cert_path, key_path):
2      if os.path.exists(cert_path) and os.path.exists(key_path):
3          with open(key_path, "rb") as f:
4              key = serialization.load_pem_private_key(f.read(), password=None)
5          with open(cert_path, "rb") as f:
6              cert = x509.load_pem_x509_certificate(f.read())
7          return cert, key
8      # chưa có → sinh Root CA self-signed mới (subject == issuer)
9      key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
10     # ... build cert + ghi file ...
11     _restrict_key_file_permissions(key_path) # chmod 0600 / icacls
12     return cert, key
13
14 def publish_root_ca_to_trust_store(root_ca_cert, trust_store_dir):
15     out_path = os.path.join(trust_store_dir, "root_ca.crt")
16     with open(out_path, "wb") as f:
17         f.write(root_ca_cert.public_bytes(serialization.Encoding.PEM))
18     return out_path

```

Phân biệt rõ khi vấn đáp: `ca_admin.py` là đường **production** (DB + AES-GCM); `core/ca.py` là đường **legacy/lab** (file + trust store). Cả hai đều sinh Root CA self-signed với cùng bộ extension.

3.5 Sinh keypair khách hàng (`customer_keys.generate_keypair`)

Mỗi khách hàng có nhiều keypair RSA. Private key cũng *encrypt-at-rest*, nhưng AAD ở đây phụ thuộc *id* của bản ghi — mà id chỉ biết sau khi INSERT. Giải pháp: insert placeholder để lấy id, rồi encrypt với AAD đúng và UPDATE lại, tất cả trong một transaction.

```

1  def _aad_for(key_id):
2      return f"customer_keys:{key_id}".encode("ascii")
3
4  def generate_keypair(owner_id, name, key_size, db_path):
5      # ... validate name + key_size ...
6      rsa_key = rsa.generate_private_key(public_exponent=65537, key_size=key_size)
7      priv_pem = _serialize_private_pem(rsa_key) # PKCS8 PEM
8      pub_pem = _serialize_public_pem(rsa_key)
9      with transaction(db_path) as conn:

```



```

10     # 1) INSERT placeholder (ct/nonce rỗng) để lấy id
11     cur = conn.execute(
12         "INSERT INTO customer_keys (owner_id, name, algorithm, key_size, "
13         " public_key_pem, encrypted_private_key, gcm_nonce, created_at) "
14         "VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
15         (owner_id, name, "RSA", key_size, pub_pem, b"", b"", now),
16     )
17     key_id = cur.lastrowid
18     # 2) Encrypt với AAD bind theo id 3) UPDATE ciphertext thật
19     nonce, ct = encrypt_blob(priv_pem, aad=_aad_for(key_id))
20     conn.execute(
21         "UPDATE customer_keys SET encrypted_private_key = ?, gcm_nonce = ? "
22         "WHERE id = ?", (ct, nonce, key_id),
23     )

```

BOLA guard: mọi hàm đọc khóa đều thêm `WHERE id=? AND owner_id=?`. Service không tin caller có đúng quyền — luôn lọc theo chủ sở hữu để chống tham chiếu đối tượng chéo (Broken Object Level Authorization / IDOR).

```

1  def load_private_key(key_id, owner_id, db_path):
2      row = conn.execute(
3          "SELECT encrypted_private_key, gcm_nonce FROM customer_keys "
4          "WHERE id = ? AND owner_id = ?", (key_id, owner_id),
5      ).fetchone()
6      if row is None:
7          raise CustomerKeyError(f"Không tìm thấy keypair id={key_id} thuộc về bạn.")
8      pem = decrypt_blob(row["gcm_nonce"], row["encrypted_private_key"],
9                          aad=_aad_for(key_id))
10     return serialization.load_pem_private_key(pem, password=None)

```

3.6 Tạo CSR theo PKCS#10 (`core/csr.build_csr`)

CSR (Certificate Signing Request) là gói “xin cấp chứng chỉ”: chứa subject, public key và SAN, được *ký bằng private key chủ nhân*. Chữ ký này là bằng chứng **proof of possession** — chứng minh người xin thực sự nắm private key tương ứng với public key trong yêu cầu.

```

1  def build_csr(private_key, common_name, san_list=None,
2               organization="X509 Demo", country="VN"):
3      common_name = common_name.strip()
4      san_values = list(san_list or [])
5      if common_name not in san_values:
6          san_values.insert(0, common_name) # TLS hiện đại: CN phải nằm trong SAN
7      subject = x509.Name([
8          x509.NameAttribute(NameOID.COUNTRY_NAME, country),
9          x509.NameAttribute(NameOID.ORGANIZATION_NAME, organization),
10         x509.NameAttribute(NameOID.COMMON_NAME, common_name),
11     ])
12     builder = (

```

```

13         x509.CertificateSigningRequestBuilder()
14         .subject_name(subject)
15         .add_extension(
16             x509.SubjectAlternativeName(_build_san_list(san_values)),
17             critical=False,
18         )
19     )
20     return builder.sign(private_key, hashes.SHA256()) # + ký = proof of possession

```

Phía duyệt CSR phải verify chữ ký trước khi cấp cert. Hàm verify dùng public key *nằm trong chính CSR* để kiểm tra:

```

1  def verify_csr_signature(csr):
2      if hasattr(csr, "is_signature_valid"):      # cryptography >= 40
3          return bool(csr.is_signature_valid)
4      try:                                       # fallback: verify thủ công
5          csr.public_key().verify(
6              csr.signature, csr.tbs_certrequest_bytes, *_verify_padding_args(csr),
7          )
8          return True
9      except InvalidSignature:
10         return False
11
12  def _verify_padding_args(csr):
13      # RSA cần padding PKCS1v15 + hash; EC chỉ cần hash
14      if isinstance(csr.public_key(), rsa.RSAPublicKey):
15          return (padding.PKCS1v15(), csr.signature_hash_algorithm)
16      return (csr.signature_hash_algorithm,)

```

3.7 Customer nộp CSR (`csr_workflow.submit_csr`)

Service customer ghép `customer_keys` với `core/csr`: kiểm tra quyền sở hữu key, validate tên miền (cho phép wildcard *.), tạo + ký CSR, lưu DB với `status='pending'`.

```

1  def _validate_common_name(cn):
2      cn = (cn or "").strip()
3      if not cn:
4          raise CSRError("Common Name không được rỗng.")
5      if len(cn) > 253:
6          raise CSRError("Common Name dài quá 253 ký tự.")
7      name_to_check = cn[2:] if cn.startswith("*.") else cn # wildcard '*. '
8      if not all(c.isalnum() or c in "-." for c in name_to_check):
9          raise CSRError("Common Name chỉ chấp nhận chữ/số/dấu '.' '-' ...")
10     return cn
11
12  def submit_csr(requester_id, customer_key_id, common_name, san_list, db_path):
13     common_name = _validate_common_name(common_name)
14     meta = get_key_meta(customer_key_id, requester_id, db_path) # BOLA guard
15     if meta is None:
16         raise CSRError(f"Keypair id={customer_key_id} không thuộc về bạn ...")

```

```

15     key = load_private_key(customer_key_id, requester_id, db_path)
16     csr = build_csr(key, common_name=common_name, san_list=san_list)
17     csr_pem = csr_to_pem(csr)
18     with transaction(db_path) as conn:
19         cur = conn.execute(
20             "INSERT INTO csr_requests (requester_id, customer_key_id, "
21             " common_name, san_list_json, csr_pem, status, submitted_at) "
22             "VALUES (?, ?, ?, ?, ?, 'pending', ?)", (...),
23         )

```

3.8 Phát hành cert (`csr_admin.approve_csr`)

Đây là transaction trung tâm của cụm. Trình tự: (1) đọc CSR và kiểm tra `status=='pending'`; (2) parse + verify chữ ký CSR; (3) load Root CA active (giải mã key); (4) build + ký cert; (5) INSERT `issued_certs` và UPDATE `csr_requests` atomic, kèm re-check chống race.

```

1  # Bước 2: parse + verify CSR (proof of possession)
2  csr_pem = bytes(row["csr_pem"])
3  csr_obj = parse_csr(csr_pem)
4  if not verify_csr_signature(csr_obj):
5      raise CSRAdminError("Chữ ký CSR không hợp lệ - có thể bị sửa hoặc khớp "
6                          "sai keypair. Từ chối phát hành.")
7
8  # Bước 3: load Root CA active (decrypt private key)
9  ca_cert, ca_key = load_active_root_ca_with_key(db_path)
10
11 # Bước 4: build + sign cert end-entity từ CSR
12 cert, serial_number = issue_cert_from_csr(
13     csr_obj, ca_cert, ca_key, validity_days=validity_days,
14     ocsp_url=ocsp_url, crl_url=crl_url,
15 )
16 cert_pem = cert.public_bytes(serialization.Encoding.PEM)

```

```

1  # Bước 5: ghi DB atomic + re-check chống race
2  with transaction(db_path) as conn:
3      cur_status = conn.execute(
4          "SELECT status FROM csr_requests WHERE id = ?", (csr_id,),
5      ).fetchone()
6      if cur_status is None or cur_status["status"] != "pending":
7          raise CSRAdminError("CSR đã được xử lý bởi admin khác. Hủy approve.")
8      conn.execute(
9          "INSERT INTO issued_certs (csr_request_id, owner_id, serial_hex, "
10         " common_name, cert_pem, not_valid_before, not_valid_after, "
11         " issued_at, issued_by) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)", (...),
12     )
13     conn.execute(
14         "UPDATE csr_requests SET status = 'approved', reviewed_at = ?, "
15         " reviewed_by = ? WHERE id = ?", (now, admin_id, csr_id),

```

16

)

- **Verify trước, ký sau:** không bao giờ ký một CSR chưa được xác thực chữ ký — nếu không, kẻ tấn công có thể sửa subject của CSR người khác và xin cert giả.
- **Re-check trong transaction:** dù đã kiểm tra pending ở đầu, vẫn kiểm tra lại *bên trong* transaction phòng trường hợp hai admin approve cùng lúc (race) — chỉ một người thắng.
- **Atomic INSERT+UPDATE:** cert và việc đổi trạng thái CSR phải cùng commit; không thể có cert được cấp mà CSR vẫn pending.

3.9 Builder cert v3 (cert_builder._build_end_entity_cert)

Hàm internal dựng cert end-entity với bộ extension cố định của TLS server cert. Nó được dùng chung cho `issue_cert_from_csr` (cấp mới) và `reissue_cert_for_renewal` (gia hạn).

```

1  builder = (
2      x509.CertificateBuilder()
3      .subject_name(subject_name)
4      .issuer_name(ca_cert.subject)          # issuer = Root CA (KHÁC subject)
5      .public_key(public_key)                # public key lấy từ CSR
6      .serial_number(x509.random_serial_number())
7      .not_valid_before(now - timedelta(minutes=1))
8      .not_valid_after(now + timedelta(days=validity_days))
9      .add_extension(x509.BasicConstraints(ca=False, path_length=None),
10                     critical=True)          # end-entity: KHÔNG phải CA
11     .add_extension(
12         x509.KeyUsage(digital_signature=True, key_encipherment=True,
13                        key_cert_sign=False, crl_sign=False, ...),
14         critical=True,
15     )
16     .add_extension(
17         x509.ExtendedKeyUsage([ExtendedKeyUsageOID.SERVER_AUTH]),
18         critical=False,                  # EKU = serverAuth (TLS server)
19     )
20 )

```

```

1  builder = (
2      builder
3      .add_extension(x509.CRLDistributionPoints([                # CRLDP
4          x509.DistributionPoint(
5              full_name=[x509.UniformResourceIdentifier(crl_url)],
6              relative_name=None, reasons=None, crl_issuer=None)],
7          critical=False)
8      .add_extension(x509.AuthorityInformationAccess([           # AIA / OCSP
9          x509.AccessDescription(
10              access_method=AuthorityInformationAccessOID.OCSP,

```

```

11         access_location=x509.UniformResourceIdentifier(ocsp_url))]),
12         critical=False)
13     .add_extension(                                # SKI
14         x509.SubjectKeyIdentifier.from_public_key(public_key), critical=False)
15     .add_extension(                                # AKI → Root CA
16         x509.AuthorityKeyIdentifier.from_issuer_public_key(ca_cert.public_key()),
17         critical=False)
18 )
19 cert = builder.sign(private_key=ca_private_key, algorithm=hashes.SHA256())

```

So sánh nhanh cấu hình cert CA và end-entity:

Extension	Root CA	End-entity (server)
BasicConstraints	ca=True, path_length=0	ca=False
KeyUsage	key_cert_sign, crl_sign	digital_signature, key_encipherment
ExtendedKeyUsage	(không)	serverAuth
SAN	(không)	DNS/IP của domain
issuer vs subject	bằng nhau (self-signed)	issuer = Root CA

`issue_cert_from_csr` chỉ là wrapper lấy subject, public key, SAN từ CSR rồi gọi `_build_end_entity_cert`; còn `reissue_cert_for_renewal` giữ nguyên subject + public key của cert cũ (admin gia hạn mà không dùng private key của khách hàng). Hàm `tamper_cert_pem` lật một bit trong vùng chữ ký để demo cert bị sửa vẫn parse được nhưng verify thất bại.

3.10 Tham số cấp phát (`system_config`)

Các tham số mặc định (key size, validity, thuật toán, URL CRL & OCSP) được lưu dạng key-value và nhúng vào cert lúc phát hành. `set_config` chỉ chấp nhận key trong whitelist để chống typo/inject.

```

1  DEFAULTS = {
2      "sig_algorithm":      "RSA-SHA256",
3      "hash_algorithm":    "SHA256",
4      "default_key_size":  "2048",
5      "default_validity_days": "365",
6      "root_ca_validity_days": "3650",
7      "prod_crl_url":      "http://localhost:8889/crl.pem",
8      "prod_ocsp_url":     "http://localhost:8888/ocsp",
9  }
10 ALLOWED_KEYS = frozenset(DEFAULTS.keys()) # whitelist chống inject

```

3.11 Toàn cảnh nhà máy sinh cert — `cert_builder.py`

`cert_builder.py` là **nhà máy** sinh *mọi* certificate trong hệ thống. Phúc *sở hữu* trọn file này; các cụm khác chỉ *gọi vào* chứ không tự dựng cert. Bảng dưới liệt kê đủ các lỗi tạo cert để không sót hàm nào, kèm việc *ai gọi* (làm rõ ranh giới với Kiệt-renew và Quân-Lab).

Hàm	Tạo ra gì	Ai gọi
<code>generate_rsa_keypair</code>	Cặp khóa RSA (chưa phải cert)	<code>submit_csr_lan</code> , Lab (Quân)
<code>issue_cert_from_csr →</code> <code>_build_end_entity_cert</code>	Cert end-entity từ CSR (luồng chính)	<code>approve_csr</code> (Phúc)
<code>create_server_cert_</code> <code>signed_by_ca</code>	Cert server do Root CA ký <i>trực tiếp</i> , không qua CSR	Verification Lab (Quân)
<code>reissue_cert_for_renewal</code>	Cấp lại cert giữ nguyên subject + public key	<code>renew_cert</code> (Kiệt)
<code>create_self_signed_cert</code>	Cert tự ký (issuer = subject) — đường legacy/test	code legacy
<code>tamper_cert_pem</code>	Lật 1 bit chữ ký để demo cert giả mạo	Verification Lab (Quân)
<code>save_cert_and_key /</code> <code>save_cert / load_cert /</code> <code>load_private_key</code>	I/O cert/key ra/từ file PEM	nhiều nơi

Table 1: Mọi lỗi tạo cert đều nằm trong `cert_builder.py` (Phúc sở hữu).

Điểm chung và khác nhau. Tất cả hàm trên đều dùng cùng một `x509.CertificateBuilder()` (builder pattern). Khác nhau ở hai chỗ:

- *Ai là issuer*: `create_self_signed_cert` và Root CA dùng `issuer == subject` (tự ký); còn `create_server_cert_signed_by_ca`, `issue_cert_from_csr`, `reissue_cert_for_renewal` dùng `issuer = ca_cert.subject` và ký bằng *private key của Root CA*.
- *Bộ extension*: cert server/end-entity có `BasicConstraints(ca=False)`, `KeyUsage` (`digitalSignature`, `keyEncipherment`), `ExtendedKeyUsage` `serverAuth`, `SAN`, `CRLDP`, `AIA`, `SKI/AKI`; còn Root CA có `ca=True`, `keyCertSign`, `crlSign`.

Trích hàm tạo server cert do Root CA ký trực tiếp (lỗi Verification Lab dùng):

```

1 def create_server_cert_signed_by_ca(server_private_key, ca_cert, ca_private_key,
2                                     common_name="localhost", dns_names=None,
3                                     validity_days=365, expired=False):
4     issuer = ca_cert.subject                # issuer = Root CA (KHÔNG
5     ↪ self-signed)
6     # ... builder: subject = CN; BasicConstraints(ca=False);
7     #     KeyUsage(digital_signature, key_encipherment); EKU serverAuth;
8     #     SAN; CRLDistributionPoints; AIA/OCSP; SKI/AKI ...

```

```

8     certificate = builder.sign(private_key=ca_private_key,    # ← Root CA ký
9                               algorithm=hashes.SHA256())
10    return certificate, serial_number

```

Vì sao có hai lỗi tạo server cert? `issue_cert_from_csr` dùng cho luồng *thật* (khách gửi CSR, admin duyệt — public key đến từ CSR). `create_server_cert_signed_by_ca` dùng cho *Verification Lab* (Quân) cần sinh nhanh nhiều cert mẫu với cả keypair do Lab tự tạo. Cả hai cùng cho ra cert end-entity hợp lệ do Root CA ký — chỉ khác nguồn gốc của public key.

3.12 Lớp UI của cụm

- `root_ca_view.py`: hiển thị Root CA active + lịch sử; nút “Sinh Root CA” / “rotate” / “Publish ra Trust Store”. Modal có cảnh báo đỏ khi rotate.
- `csr_queue_view.py`: queue duyệt CSR; `ApproveCSRDialog` gọi `approve_csr` và ghi hai audit (CSR_APPROVED + CERT_ISSUED); `RejectCSRDialog` bắt buộc nhập lý do.
- `system_config_view.py`: form 5 trường, validate int, ghi audit các thay đổi.
- `my_keys_view.py`: sinh/xem/xóa keypair (xóa bị từ chối nếu key đang được CSR tham chiếu).
- `csr_submit_view.py`: chọn keypair, nhập CN + SAN, submit CSR (offline qua DB hoặc remote qua LAN tới máy Admin).

4 Cần nắm để vấn đáp

Hỏi: Chữ ký số RSA hoạt động thế nào trong đề án? Vì sao dùng padding PKCS#1 v1.5?

Đáp: Ký = băm dữ liệu bằng SHA-256, rồi dùng private key “mã hóa” (toán RSA) lên giá trị băm đã đệm (padding). Verify = dùng public key giải ra giá trị băm, băm lại dữ liệu và so sánh. Đề án dùng padding PKCS1v15 (thấy trong `_verify_padding_args`) vì đây là sơ đồ chuẩn, tương thích rộng và đủ cho mục tiêu học thuật; PSS hiện đại hơn nhưng PKCS1v15 đơn giản, dễ giải thích. EC thì không cần padding, chỉ cần hash.

Hỏi: Root CA là self-signed — vì sao nó tự ký được và điều đó có an toàn không?

Đáp: Root CA là *gốc* của cây tin cậy, không có ai “trên” nó để ký, nên nó tự ký: `subject == issuer` và chữ ký tạo bằng *chính* private key của nó (`.sign(key, SHA256())`). Bản thân việc self-signed không tạo tin cậy — tin cậy đến từ việc cert đó được đặt vào **trust store** của client. Client tin Root CA vì “quyết định” tin, không phải vì chữ ký. An toàn nằm ở chỗ private key Root CA được bảo vệ (encrypt-at-rest) và trust store được phân phối an toàn.

Hỏi: CSR theo PKCS#10 chứa gì? Proof of possession là gì và chống được tấn công nào?

Đáp: CSR chứa subject (CN, O, C), public key, SAN, và một *chữ ký* được tạo bằng private key chủ nhân lên toàn bộ phần TBS. Verify chữ ký bằng chính public key trong CSR chính là **proof of possession** — chứng minh người gửi thực sự sở hữu private key. Nó chống tấn công “xin cert cho khóa không phải của mình”: nếu kẻ xấu lấy public key người khác và cố xin cert, hắn không thể tạo chữ ký hợp lệ vì không có private key tương ứng.

Hỏi: Khi cấp cert, các extension BasicConstraints / KeyUsage / EKU đặt thế nào và vì sao?

Đáp: Cert end-entity đặt `BasicConstraints(ca=False)` (không được ký cert khác), `KeyUsage` bật `digital_signature + key_encipherment` (đúng vai trò TLS), tắt `key_cert_sign`; và `ExtendedKeyUsage = serverAuth` để khai báo “dùng làm cert server TLS”. Ngược lại Root CA đặt `ca=True`, `path_length=0` và bật `key_cert_sign + crl_sign`. Đặt critical cho BasicConstraints và KeyUsage để client buộc phải tuân thủ.

Hỏi: Vì sao phải encrypt-at-rest private key của CA? AAD dùng để làm gì?

Đáp: Vì nếu file DB bị lộ, private key plaintext sẽ cho phép kẻ tấn công ký cert giả mạo bất kỳ — phá vỡ toàn bộ tin cậy. Mã hóa AES-256-GCM đảm bảo chỉ ai có master key mới giải được. AAD (Additional Authenticated Data) là dữ liệu được *xác thực nhưng không mã hóa*; với Root CA dùng `b"root_ca"`, với customer key dùng `b"customer_keys:{id}"`. AAD “cột” ciphertext vào đúng ngữ cảnh: nếu lấy ciphertext giải mã ở ngữ cảnh khác (AAD khác hoặc id khác), GCM tag check thất bại → chống đổi chỗ/nhầm lẫn ghi.

Hỏi: Proof of possession chống được gì cụ thể trong luồng cấp cert?

Đáp: Nó chống việc cấp cert cho một public key mà người xin không nắm private key — tức ngăn “cướp danh tính khóa”. Trong `approve_csr`, nếu `verify_csr_signature` trả về False (CSR bị sửa subject/SAN, hoặc chữ ký không khớp public key trong CSR), admin từ chối phát hành ngay. Nhờ vậy cert chỉ được cấp cho đúng chủ thực sự của cặp khóa.

Hỏi: Chuỗi renew (gia hạn) cert diễn ra thế nào? Admin có cần private key của khách hàng không?

Đáp: Gia hạn dùng `reissue_cert_for_renewal`: nó giữ *nguyên* subject và public key của cert cũ, chỉ kéo dài thời hạn và sinh serial mới, rồi ký lại bằng Root CA *đang active*. Admin **không** cần và không có private key của khách hàng — chỉ cần public key (đã nằm trong cert cũ). Đây là lý do tách `_build_end_entity_cert` dùng chung cho cả cấp mới lẫn gia hạn.

Hỏi: SKI và AKI để làm gì trong cert?

Đáp: SubjectKeyIdentifier (SKI) là vân tay của public key chủ thể; AuthorityKeyIdentifier (AKI) là vân tay của public key bên ký (Root CA). Khi client dựng chuỗi, nó khớp AKI của cert con với SKI của cert cha để nối nhanh và chính xác, kể cả khi có nhiều CA cùng tên.

5 Test case và demo

5.1 Bảng test case chính

5.2 Đoạn unit test gợi ý

```

1 def test_approve_csr_happy_path():
2     env = TestEnv(with_root_ca=True)          # đã tạo Root CA trong setup
3     csr_id = env.make_csr("demo.com", san_list=["www.demo.com"])
4     issued = approve_csr(csr_id, env.admin_id, validity_days=365,
5                           db_path=env.db_path)
6     detail = get_csr_detail(csr_id, env.db_path)
7     assert detail["status"] == "approved"
8     # issuer của cert = subject của Root CA active
9     ca_cert, _ = load_active_root_ca_with_key(env.db_path)
10    cert = x509.load_pem_x509_certificate(<cert_pem từ issued_certs>)
11    assert cert.issuer == ca_cert.subject
12    # chữ ký cert verify được bằng public key Root CA
13    ca_cert.public_key().verify(
14        cert.signature, cert.tbs_certificate_bytes,
15        padding.PKCS1v15(), cert.signature_hash_algorithm,
16    )

```

Tình huống	Kỳ vọng	File test
Tạo Root CA lần đầu	is_active=1, cert PEM parse được, subject chứa CN	test_m4_ca_admin
Private key không lưu raw vào DB	DB chỉ chứa ciphertext + nonce; decrypt mới ra key	test_m4_ca_admin
Rotate Root CA	Root CA cũ is_active=0, cái mới =1	test_m4_ca_admin
Approve CSR happy path	cert ký bởi Root CA, public key khớp CSR	test_m6_csr_admin
Approve khi chưa có Root CA	raise lỗi	test_m6_csr_admin
CSR bị sửa chữ ký	verify fail → từ chối cấp	test_m6_csr_admin
Race 2 admin approve	1 thắng, 1 fail	test_m6_csr_admin
SAN từ CSR sang cert	cert giữ đúng SAN	test_m6_csr_admin
Sinh keypair + load lại	decrypt đúng key; BOLA: owner khác load fail	test_m5_customer

```

17
18 def test_tampered_csr_rejected():
19     env = TestEnv(with_root_ca=True)
20     csr_id = env.make_csr("demo.com")
21     # giả lập sửa CSR PEM trong DB → verify_csr_signature phải fail
22     # → approve_csr raise CSRAdminError

```

5.3 Lệnh chạy test

```

1 # Chạy từng module test của cụm
2 python tests/test_m4_ca_admin.py      # Root CA admin
3 python tests/test_m6_csr_admin.py     # duyệt CSR + phát hành cert
4 python tests/test_m5_customer.py      # keypair + CSR phía customer

```

5.4 Kịch bản demo end-to-end

- Admin mở root_ca_view → “Sinh Root CA” (RSA-2048, 3650 ngày) → Publish ra Trust Store.
- Customer mở my_keys_view → “Sinh keypair mới” (RSA-2048).
- Customer mở csr_submit_view → chọn keypair, nhập example.com + SAN → Submit CSR (status pending).

4. Admin mở `csr_queue_view` → chọn CSR → “Xem chi tiết” (CSR PEM) → “Approve” (365 ngày) → cert được phát hành, status chuyển `approved`.
5. (Tùy chọn) Dùng `tamper_cert_pem` để minh họa cert bị sửa → cụm kiểm tra chuỗi sẽ FAIL verify.

6 Tích hợp vào chương trình chung

Cụm này là *nhà sản xuất* cho phần còn lại của hệ thống. Quan hệ gọi/ghép như sau:

- **UI → Service:** `root_ca_view` gọi `create_root_ca`, `publish_active_to_trust_store`; `csr_queue_view` gọi `approve_csr` / `reject_csr`; `csr_submit_view` gọi `submit_csr`; `my_keys_view` gọi `generate_keypair`.
- **Service → Core:** `csr_admin.approve_csr` ghép ba mảnh: `core/csr.verify_csr_signature`, `ca_admin.load_active_root_ca_with_key` và `cert_builder.issue_cert_from_csr`.
- **Service → Config:** validity/URL CRL & OCSP lấy từ `system_config` để nhúng vào cert; URL còn fallback về `infra_manager` để tự khớp port runtime.
- **Đầu ra cho cụm khác:** cert trong `issued_certs` và Root CA trong trust store là input cho cụm **kiểm tra chuỗi** (`core/verify.py`); serial của cert là input cho cụm **thu hồi CRL & OCSP**.
- **Audit:** mọi hành động nhạy cảm (tạo/rotate Root CA, approve/reject CSR, cấp cert, sinh keypair, đổi config) đều ghi log qua `services.audit.write_audit`.

Sợi chỉ xuyên suốt: **một chữ ký số ở mỗi mắt xích**. CSR mang chữ ký khách hàng (proof of possession) → cert mang chữ ký Root CA (chứng thực) → Root CA tự ký (gốc tin cậy). Năm sợi chỉ này là năm cả cụm.

7 Bổ sung sau rà soát: các hàm/feature dễ bị hỏi

7.1 Nộp CSR qua LAN & khái niệm khóa “public-only”

Câu hỏi điển hình: “Nộp CSR từ máy khác thì private key có bị gửi đi không?” — KHÔNG. Ở chế độ **remote (LAN)**: client `build_csr` tại máy mình, chỉ gửi CSR PEM + username/password tới Admin API. Máy Admin lưu thành keypair “**public-only**” (cột `encrypted_private_key` rỗng).

- Hai chế độ submit: *offline* (`submit_csr` ghi thẳng DB, key nằm trong DB cùng máy) vs *remote LAN* (key ở lại client).

- Hệ quả cờ `is_public_only`: `list_keys/get_key_meta` gắn cờ này; `load_private_key` sẽ *raise* với key public-only (“private key nằm trên máy client gốc”) — không thể ký bằng key không có.

7.2 Các hàm CSR/CA/cấu hình còn lại

- `cancel_csr` (**customer tự hủy**). Chỉ hủy được CSR `pending`; thực chất set `status='rejected'`, `reason='cancelled by requester'` (giữ lịch sử thay vì xóa cứng), có guard `WHERE requester_id=?`; bị chặn ở chế độ remote (CSR nằm trên máy Admin).
- `list_my_csr` / `get_my_csr_by_id` (**BOLA**). Customer chỉ thấy CSR của mình (lọc `requester_id`) — đối lập với `list_all_csr/get_csr_detail` của admin (xem MỌI CSR vì là quyền quản trị).
- `get_active_root_ca` vs `load_active_root_ca_with_key`. `get_active_root_ca` chỉ đọc `metadata+cert_pem`, *KHÔNG* giải mã private key (giảm bề mặt lộ khóa); chỉ `load_active_root_ca_with_key` mới decrypt — và chỉ khi cần *ký*. `list_root_ca_history` liệt kê CA active+retired cho bảng lịch sử rotate.
- **Đọc cấu hình**. `get_config/get_all_config` đọc giá trị; `get_int_config` có *fallback* an toàn khi thiếu/parse lỗi; `seed_defaults` idempotent lúc init DB. UI lấy default `key_size/validity` qua `get_int_config`; `csr_admin._default_crl_url/_default_ocsp_url` ưu tiên config rồi fallback `infra_manager`.
- **Trích thông tin CSR & trust store**. `get_csr_common_name` (CN, trả "" nếu trống) và `get_csr_san_dns` (list DNS-SAN, rỗng nếu thiếu) — dùng hiển thị/đối chiếu khi duyệt. `ca.load_trust_store` đọc mọi `.crt/.pem` trong trust store (bỏ qua file PEM hỏng thay vì crash), thiết kế list để mở rộng nhiều Root CA — là đầu vào cho cụm verify.

8 Checklist chuẩn bị vấn đáp

- ☐ Giải thích được tại sao Root CA self-signed (`subject==issuer`, tự ký bằng khóa của nó) và tin cậy đến từ trust store.
- ☐ Đọc trôi bộ extension Root CA: `BasicConstraints` (`ca=True`, `path_length=0`), `KeyUsage` (`key_cert_sign`, `crl_sign`), `SKI`.
- ☐ Phân biệt cert CA và end-entity qua bảng so sánh extension (`BasicConstraints`, `KeyUsage`, `EKU`, `SAN`).
- ☐ Trình bày proof of possession: CSR ký bằng khóa chủ, admin verify trước khi cấp.
- ☐ Giải thích encrypt-at-rest (AES-256-GCM) và vai trò của AAD (`root_ca` / `customer_keys:{id}`).

- ☐ Mô tả 5 bước của `approve_csr` và lý do verify-trước-ký-sau + re-check chống race + atomic.
- ☐ Nói được BOLA guard: luôn `WHERE owner_id=?`.
- ☐ Giải thích chữ ký RSA + padding PKCS#1 v1.5 + SHA-256.
- ☐ Trình bày chuỗi renew dùng `reissue_cert_for_renewal` (giữ public key, admin không cần private key khách hàng).
- ☐ Cảnh báo rotate Root CA làm cert cũ FAIL verify đến khi re-issue.
- ☐ Chạy được 3 file test (`m4`, `m6`, `m5`) và demo end-to-end tạo Root CA → cấp cert.